

UNIVERSIDAD TECNOLÓGICA NACIONAL

Tecnicatura Universitaria en Programación a Distancia
Programación 1

Trabajo Práctico Integrador
Gestión de Datos de Países en Python:
Filtros, Ordenamientos y Estadísticas



Alumno:
Zacarías Cabral

Año Lectivo

2026

Índice

1. Introducción
2. Objetivos del Proyecto
3. Marco Teórico
 - 3.1 Listas
 - 3.2 Diccionarios
 - 3.3 Funciones
 - 3.4 Estructuras Condicionales
 - 3.5 Estructuras Repetitivas
 - 3.6 Ordenamientos
 - 3.7 Estadísticas Básicas
 - 3.8 Archivos CSV
4. Diseño y Arquitectura del Sistema
5. Desarrollo del Proyecto
6. Resultados Obtenidos
7. Dificultades Encontradas
8. Conclusiones
9. Bibliografía
10. Anexos

1. Introducción

En el presente Trabajo Práctico Integrador se desarrolló una aplicación en Python destinada a la gestión de información de países. El sistema permite almacenar, consultar, filtrar, ordenar y analizar datos relacionados con distintos países utilizando estructuras de datos vistas durante la cursada.

El proyecto fue desarrollado aplicando conceptos fundamentales de Programación 1, tales como listas, diccionarios, funciones, estructuras condicionales, estructuras repetitivas y manejo de archivos CSV.

Además de implementar las funcionalidades solicitadas, se trabajó sobre la modularización del código mediante la separación en distintos archivos Python, favoreciendo la organización, reutilización y mantenimiento del sistema.

2. Objetivos del Proyecto

Objetivo General

Desarrollar una aplicación en Python que permita gestionar información de países mediante búsquedas, filtros, ordenamientos y estadísticas utilizando estructuras de datos y archivos CSV.

Objetivos Específicos

- Aplicar listas y diccionarios para representar información estructurada.
- Implementar funciones para modularizar el programa.
- Utilizar estructuras condicionales y repetitivas.
- Leer y escribir información en archivos CSV.
- Aplicar filtros y ordenamientos sobre conjuntos de datos.
- Obtener estadísticas básicas a partir de los datos almacenados.
- Organizar el proyecto mediante módulos independientes.

3. Marco Teórico

3.1 Listas

Las listas son estructuras de datos que permiten almacenar múltiples elementos dentro de una misma variable.

En Python, las listas se representan mediante corchetes [] y permiten agregar, eliminar, modificar y recorrer elementos.

En este proyecto se utilizó una lista para almacenar todos los países cargados desde el archivo CSV. Cada elemento de la lista representa un país.

Ejemplo:

```
países = [  
    {"nombre": "Argentina", "poblacion": 46000000},  
    {"nombre": "Brasil", "poblacion": 214000000}  
]
```

Gracias al uso de listas fue posible recorrer los países, realizar búsquedas, aplicar filtros, ordenar información y calcular estadísticas.

3.2 Diccionarios

Los diccionarios son estructuras de datos que almacenan información mediante pares clave-valor.

Cada dato se identifica mediante una clave única que permite acceder fácilmente a la información asociada.

En este proyecto cada país fue representado mediante un diccionario que contiene su nombre, población, superficie y continente.

Ejemplo:

```
{  
    "nombre": "Argentina",  
    "poblacion": 46000000,  
    "superficie": 2780400,  
    "continente": "America"  
}
```

El uso de diccionarios permitió organizar la información de forma clara y acceder fácilmente a cada dato cuando se realizaron búsquedas, filtros y estadísticas.

3.3 Funciones

Las funciones son bloques de código diseñados para realizar una tarea específica.

Su principal ventaja es evitar la repetición de código y mejorar la organización de los programas.

En este proyecto se utilizó una función para cada responsabilidad del sistema, por ejemplo:

- `cargar_csv()`
- `mostrar_paises()`
- `buscar_pais()`
- `agregar_pais()`
- `actualizar_pais()`
- `ordenar_nombre()`
- `mostrar_estadisticas()`

Gracias a las funciones fue posible dividir el proyecto en módulos más pequeños, facilitando la lectura, el mantenimiento y la reutilización del código.

3.4 Estructuras Condicionales

Las estructuras condicionales permiten que un programa tome decisiones según determinadas condiciones.

En Python se utilizan principalmente las instrucciones `if`, `elif` y `else`.

En este proyecto fueron utilizadas para validar datos ingresados por el usuario, controlar opciones del menú y verificar distintas situaciones, como la existencia de un país o la validez de una búsqueda.

Ejemplo:

```
if poblacion <= 0:  
    print("Error: la población debe ser mayor que cero")
```

Gracias a las estructuras condicionales fue posible controlar el flujo del programa y evitar datos incorrectos.

3.5 Estructuras Repetitivas

Las estructuras repetitivas permiten ejecutar un conjunto de instrucciones varias veces.

En Python las más utilizadas son `for` y `while`.

En este proyecto se utilizaron para recorrer la lista de países, mostrar información, realizar búsquedas, calcular estadísticas y mantener activo el menú principal hasta que el usuario decidiera salir del sistema.

Ejemplo:

```
for pais in paises:  
    print(pais["nombre"])
```

También se utilizaron ciclos **while** para validar el ingreso de datos y repetir solicitudes cuando el usuario ingresaba valores incorrectos.

Gracias a estas estructuras fue posible procesar grandes cantidades de información de forma eficiente.

3.6 Ordenamientos

Los algoritmos de ordenamiento permiten organizar los datos siguiendo un criterio determinado.

En este proyecto se implementaron ordenamientos por:

- Nombre
- Población
- Superficie

Para ello se utilizó la función **sorted()** de Python, la cual genera una nueva lista ordenada sin modificar la lista original.

Ejemplo:

```
países_ordenados = sorted(paises, key=obtener_nombre)
```

Además, para el ordenamiento por superficie se permitió elegir entre orden ascendente y descendente mediante el parámetro `reverse=True`.

Los ordenamientos facilitan la visualización y el análisis de la información almacenada en el sistema.

3.7 Estadísticas Básicas

Las estadísticas básicas permiten obtener información resumida a partir de un conjunto de datos.

En este proyecto se implementaron las siguientes estadísticas:

- País con mayor población.
- País con menor población.
- Promedio de población.
- Promedio de superficie.
- Cantidad de países por continente.

Para calcular estos indicadores se recorrió la lista de países utilizando ciclos y variables acumuladoras.

Ejemplo:

```
promedio_poblacion = suma_poblacion / len(paises)
```

Las estadísticas permiten analizar la información almacenada y obtener resultados útiles para la toma de decisiones.

3.8 Archivos CSV

Un archivo CSV (Comma Separated Values) es un archivo de texto utilizado para almacenar datos organizados en filas y columnas.

Cada fila representa un registro y cada columna representa un dato específico.

En este proyecto se utilizó un archivo CSV para almacenar la información de los países, permitiendo conservar los datos incluso después de cerrar el programa.

Ejemplo:

```
nombre,poblacion,superficie,continente  
Argentina,46000000,2780400,America  
Brasil,214000000,8515767,America
```

La lectura y escritura de los datos se realizó mediante la librería csv de Python.

El uso de archivos CSV permitió implementar persistencia de datos, facilitando la carga inicial de información y el almacenamiento de los cambios realizados por el usuario.

4. Diseño y Arquitectura del Sistema

Para el desarrollo del proyecto se utilizó una estructura modular, dividiendo el sistema en distintos archivos Python según la responsabilidad de cada uno.

Esta organización permitió mantener el código más ordenado, facilitar las pruebas y simplificar el mantenimiento del proyecto.

Estructura de Archivos

```
TP_Paises/  
├── main.py  
├── carga_datos.py  
├── busquedas.py  
├── filtros.py  
├── ordenamientos.py  
├── estadisticas.py  
├── paises.csv  
└── README.md
```

Descripción de los Módulos

main.py

Es el punto de entrada del sistema. Contiene el menú principal y coordina la ejecución de todas las funcionalidades.

carga_datos.py

Contiene las funciones encargadas de leer y guardar la información en el archivo CSV, además de las funciones para mostrar, agregar y actualizar países.

busquedas.py

Incluye las funciones de búsqueda de países por nombre, permitiendo coincidencias exactas y parciales.

filtros.py

Contiene los filtros por continente, rango de población y rango de superficie.

ordenamientos.py

Incluye las funciones para ordenar países por nombre, población y superficie.

estadisticas.py

Calcula y muestra las estadísticas generales del sistema.

Flujo General del Programa

Al iniciar la aplicación se cargan los datos almacenados en el archivo CSV.

Posteriormente se muestra un menú principal que permite al usuario seleccionar distintas operaciones sobre los países almacenados.

Las acciones realizadas por el usuario modifican la lista de países en memoria y, cuando corresponde, los cambios son guardados nuevamente en el archivo CSV para conservar la información.

Esta arquitectura permitió separar claramente las responsabilidades de cada módulo y mejorar la organización general del proyecto.

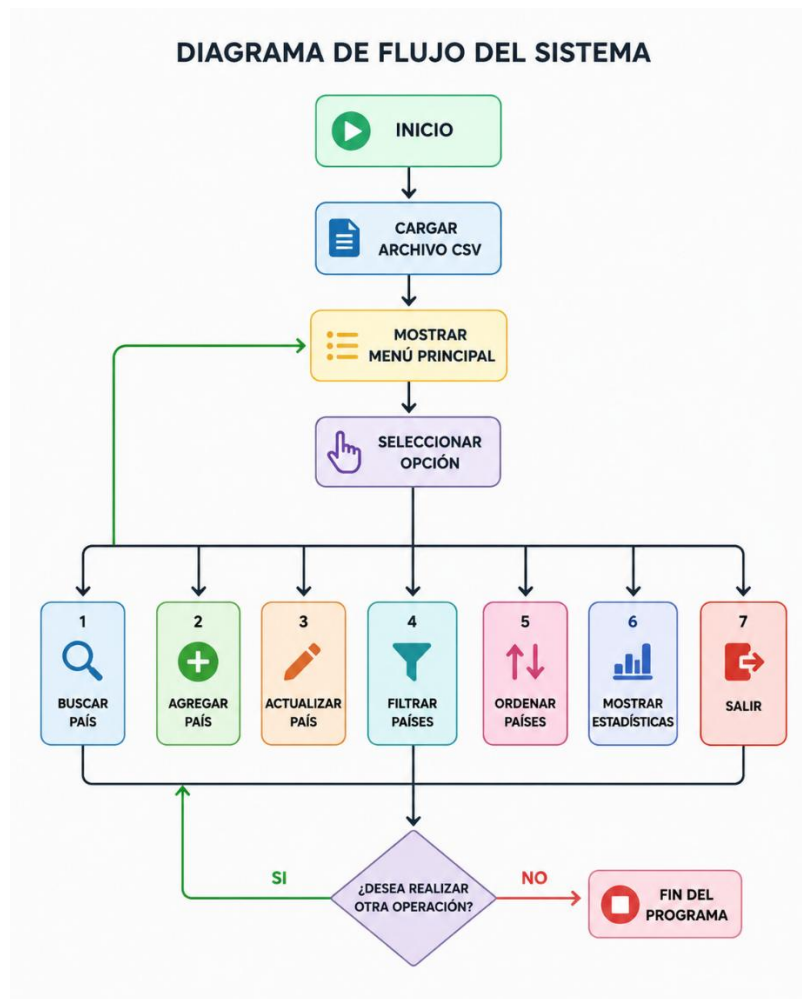


Figura 1. Diagrama de flujo general del sistema de gestión de países desarrollado en Python.

5. Desarrollo del Proyecto

Para el desarrollo de este trabajo se utilizó Python como lenguaje de programación y un archivo CSV para almacenar la información de los países.

La información fue organizada mediante una lista de diccionarios, donde cada diccionario representa un país y contiene los siguientes datos:

- Nombre
- Población
- Superficie
- Continente

Ejemplo de estructura utilizada:

```
{  
  "nombre": "Argentina",  
  "poblacion": 46000000,  
  "superficie": 2780400,  
  "continente": "America"  
}
```

Carga de Datos

La aplicación comienza leyendo la información almacenada en el archivo `países.csv`.

Para ello se desarrolló la función `cargar_csv()`, que utiliza la librería `csv` de Python para leer cada registro y convertirlo en un diccionario.

Los países cargados son almacenados en una lista que será utilizada durante toda la ejecución del programa.

Gestión de Países

Se implementaron funciones para permitir la administración de los registros:

- Mostrar países.
- Buscar países por nombre.
- Agregar nuevos países.
- Actualizar información existente.

Durante el ingreso de datos se realizaron validaciones para evitar campos vacíos y controlar que los valores numéricos sean correctos.

Filtros

El sistema permite realizar consultas específicas mediante filtros.

Se implementaron filtros por:

- Continente.
- Rango de población.
- Rango de superficie.

Estas funciones permiten mostrar únicamente los países que cumplen determinadas condiciones.

Ordenamientos

Para facilitar la visualización de la información se desarrollaron distintos ordenamientos utilizando la función `sorted()`.

Los criterios implementados fueron:

- Nombre.
- Población.
- Superficie.

En el caso de la superficie se agregó la posibilidad de ordenar de forma ascendente o descendente.

Estadísticas

Se implementó una función encargada de calcular estadísticas generales del conjunto de países.

Entre los indicadores obtenidos se encuentran:

- País con mayor población.
- País con menor población.
- Promedio de población.
- Promedio de superficie.
- Cantidad de países por continente.

Para ello se utilizaron recorridos sobre la lista de países y variables acumuladoras.

Persistencia de Datos

Con el objetivo de conservar los cambios realizados por el usuario, se desarrolló la función `guardar_csv()`.

Esta función permite escribir nuevamente todos los registros en el archivo CSV cada vez que se agrega o actualiza un país.

De esta manera los datos permanecen almacenados incluso después de cerrar el programa.

Manejo de Errores

Se incorporaron validaciones y manejo básico de excepciones para evitar errores durante la ejecución.

Entre ellas se incluyen:

- Control de valores numéricos inválidos.
- Validación de campos vacíos.
- Verificación de países existentes.
- Manejo de ausencia del archivo CSV mediante `FileNotFoundException`.

Estas validaciones mejoran la robustez del sistema y brindan mensajes claros al usuario.

6. Resultados Obtenidos

Una vez finalizado el desarrollo, se realizaron distintas pruebas para verificar el correcto funcionamiento de todas las funcionalidades solicitadas en la consigna.

Durante las pruebas se comprobó que el sistema fue capaz de:

- Leer correctamente los datos almacenados en el archivo CSV.
- Mostrar la información de los países cargados.
- Buscar países mediante coincidencias exactas y parciales.
- Agregar nuevos países al sistema.
- Actualizar los datos de población y superficie.
- Filtrar países por continente.
- Filtrar países por rango de población.
- Filtrar países por rango de superficie.
- Ordenar países por nombre, población y superficie.
- Calcular estadísticas generales.
- Guardar automáticamente los cambios realizados en el archivo CSV.

Además, se verificó el correcto funcionamiento de las validaciones implementadas, evitando el ingreso de datos inválidos y mostrando mensajes claros al usuario.

Evidencias de Funcionamiento

A continuación, se presentan capturas de pantalla correspondientes a distintas ejecuciones del sistema:

- Menú principal.
- Búsqueda de países.
- Agregado de un nuevo país.
- Actualización de información.

- Aplicación de filtros.
- Ordenamiento de registros.

Estas pruebas permitieron comprobar que el sistema cumple con los requerimientos establecidos en la consigna del trabajo práctico.

Evaluación General

Los resultados obtenidos fueron satisfactorios, ya que todas las funcionalidades principales fueron implementadas y probadas correctamente.

```

File Edit Selection View Go Run ... TP_Paises
EXPLORER
TP_PAISES
  __pycache__
  busquedas.cpython-...
  carga_datos.cpython-...
  estadisticas.cpython-...
  filtros.cpython-314.pyc
  ordenamiento.cpython-...
  busquedas.py
  carga_datos.py
  estadisticas.py
  filtros.py
  main.py
  ordenamiento.py
  pais.csv
  README.md
  OUTLINE
  TIMELINE
main.py
18 1. Mostrar pais
19 2. Buscar pais
20 3. Agregar pais
21 4. Actualizar pais
22 5. Filtrar por continente
23 6. Filtrar por poblacion
24 7. Filtrar por superficie
25 8. Ordenar por nombre
26 9. Ordenar por poblacion
27 10. Ordenar por superficie
28
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Usuario\Desktop\TP_Paises> & C:/Users/Usuario/AppData/Local/Programs/Python/Python314/python.exe c:/Users/Usuario/Desktop/TP_Paises/main.py
MENU PRINCIPAL
1. Mostrar pais
2. Buscar pais
3. Agregar pais
4. Actualizar pais
5. Filtrar por continente
6. Filtrar por poblacion
7. Filtrar por superficie
8. Ordenar por nombre
9. Ordenar por poblacion
10. Ordenar por superficie
11. Mostrar estadisticas
12. Salir

```

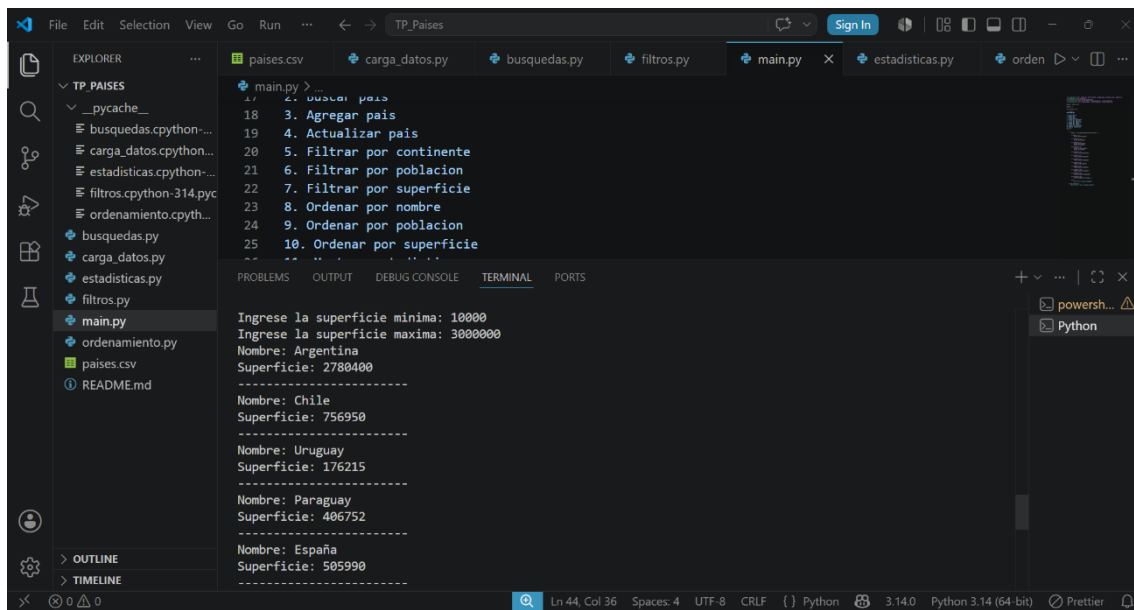
Captura1. Menú Principal

```

File Edit Selection View Go Run ... TP_Paises
EXPLORER
TP_PAISES
  __pycache__
  busquedas.cpython-...
  carga_datos.cpython-...
  estadisticas.cpython-...
  filtros.cpython-314.pyc
  ordenamiento.cpython-...
  busquedas.py
  carga_datos.py
  estadisticas.py
  filtros.py
  main.py
  ordenamiento.py
  pais.csv
  README.md
  OUTLINE
  TIMELINE
main.py
18 1. Mostrar pais
19 2. Buscar pais
20 3. Agregar pais
21 4. Actualizar pais
22 5. Filtrar por continente
23 6. Filtrar por poblacion
24 7. Filtrar por superficie
25 8. Ordenar por nombre
26 9. Ordenar por poblacion
27 10. Ordenar por superficie
28
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Seleccione una opcion: 1
Nombre: Argentina
Poblacion: 46000000
Superficie: 2780400
Continente: America
-----
Nombre: Brasil
Poblacion: 214000000
Superficie: 8515767
Continente: America
-----
Nombre: Chile
Poblacion: 19000000
Superficie: 756950
Continente: America
-----
Nombre: Uruguay

```

Captura 2. Mostrar Países



```
main.py >
1. Buscar pais
18 2. Agregar pais
19 3. Actualizar pais
20 4. Filtrar por continente
21 5. Filtrar por poblacion
22 6. Filtrar por superficie
23 7. Ordenar por nombre
24 8. Ordenar por poblacion
25 9. Ordenar por superficie
10. Ordenar por superficie

Ingrese la superficie mínima: 10000
Ingrese la superficie máxima: 3000000
Nombre: Argentina
Superficie: 2780400
-----
Nombre: Chile
Superficie: 756950
-----
Nombre: Uruguay
Superficie: 176215
-----
Nombre: Paraguay
Superficie: 406752
-----
Nombre: España
Superficie: 505990
-----
```

Captura 3. Opción 7, filtrar por superficie.

7. Dificultades Encontradas

Durante el desarrollo del proyecto, una de las principales dificultades fue trabajar con archivos CSV, ya que era la primera vez que se utilizaba este tipo de almacenamiento de datos en un proyecto práctico.

También fue necesario comprender cómo organizar el código en distintos módulos para lograr una correcta modularización del sistema.

Estas dificultades fueron resueltas mediante pruebas, consultas a la documentación y la aplicación progresiva de los conceptos trabajados durante la cursada.

8. Conclusiones

Este trabajo permitió integrar y aplicar los principales conceptos desarrollados en la materia Programación 1.

A través de la implementación del sistema se trabajó con listas, diccionarios, funciones, estructuras de control, archivos CSV, filtros, ordenamientos y estadísticas.

Además, se comprendió la importancia de la modularización y de la organización del código para desarrollar aplicaciones más claras y fáciles de mantener.

En conclusión, el proyecto resultó una experiencia útil para consolidar los conocimientos adquiridos durante la cursada y aplicarlos en un caso práctico real.

9. Bibliografía

- Python Software Foundation. Python Documentation.
<https://docs.python.org/3/>
- Python CSV File Reading and Writing.
<https://docs.python.org/3/library/csv.html>
- Material de estudio de Programación 1. Tecnicatura Universitaria en Programación a Distancia (UTN).
- Apuntes y actividades prácticas desarrolladas durante la cursada.
- Datos sobre los países añadidos en el CSV: Google

10. ANEXOS

 Repositorio GitHub

https://github.com/ZacariasC/TPI_Programacion1_Paises.git

 Video Demostrativo

<https://youtu.be/rZwG4SuRmjw>

